BMS INSTITUTE OF TECHNOLOGY & MANAGEMENT

(An Autonomous Institution affiliated to VTU, Belagavi)
Yelahanka, Bengaluru-560 119

C Programming Laboratory		Semester	I/II
Course Code	**1BPICL106**	CIE Marks	50
Teaching Hours/Week (L:T:P:S)	0:0:2:0	SEE Marks	50
Total Hours of Pedagogy	2/week	Total Marks	100
Credits	01	Exam Hours	3
Scheme	2025	Academic Year	2025-26
Examination type (SEE)	**Practical**		

Course outcome (Course Skill Set)

At the end of the course, the student will be able to:

CO1: Apply the concepts of Algorithm, flowchart and basic C constructs to solve the problems

CO2: Make use of arrays & strings with modular programming concepts to solve problems.

CO3: Apply the concepts of Pointers and Structures for real world scenarios.

CO4: Develop C programs to solve practical problems, employing core programming principles and debugging techniques.

CO5: Analyse the code snippets to identify the errors and predict the output.

PART – A
FIXED SET OF EXPERIMENTS

1.Write a C program to **compute the roots of a quadratic equation** by accepting the coefficients **a**, **b**, and **c** from the user. The program should calculate the discriminant and determine the nature of the roots (real and distinct, real and equal, or complex), then compute and display the roots with appropriate messages.

Input: Three space-separated floating-point numbers representing the coefficients a, b, and c ($-10^3 \le a, b, c \le 10^3$ and $a \ne 0$).

Output: A message indicating the type of roots, followed by the computed roots rounded to two decimal places. If the roots are complex, display both real and imaginary parts.

2.You are developing a billing software module for an electricity supply company **"Green Volt Power Services"**. The program will help the company's customer service department quickly calculate monthly electricity bills for individual customers.

The billing follows the company's tariff policy:

i) **First 200 units:** ₹0.80 per unit
ii) **Next 100 units (201–300):** ₹0.90 per unit
iii) **Beyond 300 units:** ₹1.00 per unit
iv) **Minimum meter charge:** ₹100 for all customers (regardless of usage)
v) **Surcharge:** If the energy cost (excluding meter charge) exceeds ₹400, add a **15% surcharge** on that cost.

Input: A string representing the **customer's name** (no spaces)
Output: An integer representing the **units consumed**

3.The bank's security system needs to verify cheque numbers entered by customers. A cheque number is considered unusual if it is a **palindrome** Such numbers are rare and may be flagged for a **special security check** as they are often used for memorable or special transactions.
Input : A single positive integer n ($1 \leq n \leq 10^6$)
Output: It is a palindrome or It is not a palindrome

4. In a biometrics research lab, scientists often need to calculate the number of possible arrangements of fingerprints in a dataset. The number of arrangements for n unique fingerprints is given by the factorial of n. To make the process efficient and easier to maintain, they decide to implement the factorial calculation using a recursive function in C.
Input: A single non-negative integer n ($0 \leq n \leq 20$).
Output: A single line displaying Factorial of n = <value>.

5. In a **university library**, all book IDs are stored in ascending order in the database.
When a student enters a book ID to search, the system must **quickly** check if the book exists in the catalog and display its **position** in the list if found. If the book ID does not exist, display a message indicating the book is not available.
Input:
- First line: Integer n (number of books in the catalog, $1 \leq n \leq 10^5$)
- Second line: n space-separated sorted integers (book IDs)
- Third line: Integer key (the book ID to search)

Output:
- Book found at position <index> (0-based index) if the book exists.
- Book not found if it doesn't exist.

6. The central branch of **Secure Bank** receives daily transaction amounts from its various ATMs.
Before generating end-of-day reports, the transaction amounts must be sorted in ascending order for easier auditing and anomaly detection.
Input:
- The first line contains an integer n ($1 \leq n \leq 10^4$) — the number of transactions.
- The second line contains n space-separated integers representing the transaction amounts.

Output:
- A single line displaying the sorted transaction amounts in ascending order, separated by spaces.

7**.** In the TechText Data Processing Unit, software tools handle large volumes of text data received from different departments. However, some systems operate in low-memory embedded environments where built-in string library functions like strcmp, strcat, and strlen are not available. To ensure compatibility, engineers need to manually implement basic string operations using custom functions.
The operations required are:
1. String Comparison – to check if two pieces of data are identical.
2. String Concatenation – to merge two text inputs for further processing.
3. String Length Calculation – to find the size of incoming text data.

Input:
- First line: an integer choice (1 for Compare, 2 for Concatenate, 3 for Length)
- For choice 1 or 2: read two strings str1 and str2
- For choice 3: read one string str

Output:

- For Compare: print "Strings are equal" or "Strings are not equal"
- For Concatenate: print the concatenated string
- For Length: print "Length = <value>"

8. In a **railway reservation system**, the seat allocation data for each train is stored in a matrix format.

- The first matrix contains the **number of seats booked** for each coach on different routes.
- The second matrix contains the **fare per seat** for each coach type on each route.

To calculate the **total fare collected** for every coach-route combination, the system multiplies these two matrices. Write a program using functions to perform this matrix multiplication. The program should:

- Read the dimensions and elements of two matrices.
- Verify that the number of columns in the first matrix equals the number of rows in the second matrix (matrix multiplication rule).
- if valid, multiply the matrices and display the resulting matrix.
- If invalid, display "Matrix multiplication not possible".

Input:

- First line: two integers m1 and n1 ($1 \leq m1, n1 \leq 100$), the dimensions of the first matrix.
- Next m1 lines: each containing n1 space-separated integers (booked seats matrix).
- Next line: two integers m2 and n2 ($1 \leq m2, n2 \leq 100$), the dimensions of the second matrix.
- Next m2 lines: each containing n2 space-separated integers (fare per seat matrix).

Output:

- If multiplication is possible (n1 == m2), print the product matrix with m1 rows and n2 columns, each row on a new line with space-separated integers.
- Otherwise, print "Matrix multiplication not possible".

9. Use structures and write a program to manage performance data for cricket academy players, storing their ID, name, score, and skill level. The program should read details for N players, compute the overall average score, assign skill levels based on performance ranges, display the average and category-wise counts, list all players with their details, and identify the top three performers.

Input:

The first line contains an integer N representing the number of players ($1 \leq N \leq 100$).

The next N lines each contain three values: an integer player_id (unique player ID), a string player_name (the player's full name, underscores allowed for spaces), and an integer score (runs scored in the match, $0 \leq score \leq 100$).

Output:

The program should display:

1. The overall average score (rounded to two decimal places).
2. The number of players in each skill level category (Elite, Advanced, Intermediate, Beginner).
3. A detailed list of all players showing ID, name, score, and skill level.
4. The top three performers with their names and scores in descending order.

10. In a **manufacturing quality control system**, engineers measure the **lengths** of n machine parts in millimeters to ensure they meet design specifications.To analyze production consistency, the system must compute:

1. The **total length** of all measured parts.

2. The **average length** (mean).
3. The **standard deviation** to determine variation in part sizes.

Write a C program using **pointers** to store the measurements in an array and perform all calculations using **pointer arithmetic**.

Input:

- First line: An integer n ($1 \leq n \leq 1000$) — number of parts measured.
- Second line: n space-separated real numbers representing the lengths of the parts (in mm).

Output:

- Sum = <value>
- Mean = <value>
- Standard Deviation = <value>(All values rounded to two decimal places.

PART – B

OPEN ENDED EXPERIMENTS

Open-ended experiments are a type of laboratory activity where the outcome is not predetermined and students are given the freedom to explore, design, and conduct the experiment based on the problem statements as per the concepts defined by the course coordinator. It encourages creativity, critical thinking, and inquiry-based learning.

1. Control structures
2. Arrays, Strings, Functions
3. Structures, Pointers

Suggested Learning Resources: (Textbook/ Reference Book/ Manuals):

Text books:

- Computer Science : A Structured Programming Approach Using C, by Behrouz A. Forouzan, Richard F .Gilberg, Third Edition, Cengage India Private Limited, ISBN 9788131503638, January 2007.

Reference books / Manuals:

- Brian W. Kernighan and Dennis M. Ritchie, **"The 'C' Programming Language"**, Prentice Hall of India.
- Computer fundamentals and programming in c, **"Reema Thareja",** Oxford University, Second edition, 2017.
- Jeff Szuhay , "Learn C Programming" Pact Publishing,June 2020.

Web links and Video Lectures(e-Resources):

1. elearning.vtu.ac.in/econtent/courses/video/BS/15PCD23.html
2. **Introduction to Programming in C** [https://onlinecourses.nptel.ac.in/noc23_cs02/preview]
3. **C for Everyone: Programming Fundamentals** [https://www.coursera.org/learn/c-for-everyone]
4. **Computer Programming Virtual Lab** [https://cse02-iiith.vlabs.ac.in/exp/pointers/]
5. **C Programming: The ultimate way to learn the fundamentals of the C language** [https://www.pdfdrive.com/c-programming-the-ultimate-way-to-learn-the-fundamentals-of-the-c-language-e187584209.html]
6. **C Programming: The Complete Reference** [https://viden.io/knowledge/programming-in-c-language/attachment/28313/c-the-complete-reference-herbert-schildt-4th-edition-pdf/preview]

Teaching-Learning Process (Innovative Delivery Methods):
The following are sample strategies that educators may adopt to enhance the effectiveness of the teaching-learning process and facilitate the achievement of course outcomes.

1. 1. Flipped Classroom
2. Interactive Coding Platforms

Assessment Structure:
The assessment for each course is equally divided between Continuous Internal Evaluation (CIE) and the Semester End Examination (SEE), with each component carrying **50% weightage** (i.e., 50 marks each).

The CIE marks awarded shall be based on the continuous evaluation of the laboratory report using a defined set of rubrics. Each experiment report can be evaluated for 30 marks. The laboratory test (duration 03 hours) at the end of the last week of the semester /after completion of all the experiments (whichever is early) shall be conducted for 50 marks and scaled down to 20 marks. For both CIE and SEE, the student is required to conduct one experiment each from both Part A and Part B.

Rubrics for CIE – Continuous assessment:

Rubrics for Practical continuous assessment				
Performance Indicators	**Excellent**	**Very Good**	**Good**	**Satisfactory**
Fundamental Knowledge (4) (PO1)	The student has well depth knowledge of the topics related to the course (4)	Student has good knowledge of some of the topics related to course (3)	Student is capable of narrating the answer but not capable to show in depth knowledge (2)	Student has not understood the concepts clearly (1)
Design Of Experiment (5) (PO2 & PO3)	Student is capable of discussing more than one design for his/her problem statement and capable of proving the best suitable design with proper reason (5)	Student is capable of discussing few designs for his/her problem statement but not capable of selecting best (4)	Student is capable of discussing single design with its merits and de-merits (3)	Student is capable of explaining the design (1-2)
Implementatio n (8) (PO3 & PO8)	Student is capable of implementing the design with best suitable algorithm considering optimal solution. (7-8)	Student is capable of implementing the design with best suitable algorithm and should be capable of explaining it (5-6)	Student is capable of implementing the design with proper explanation. (3-4)	Student is capable of implementing the design. (1-2)
Result &Analysis (5) (PO4)	Student is able to run the program on various cases and compare the result with proper analysis. (5)	Student will be able to run the program for all the cases. (4)	Student will be able to run the code for few cases and analyze the output. (3)	Student will be able to run the program but not able to analyze the output. (1-2)
Demonstration (8) (PO9)	The lab record is well-organized, with clear sections (e.g., Introduction, Method, Results, Conclusion).	The lab record is organized, with clear sections, but some sections are not well-defined. (5-6)	The lab record lacks clear organization or structure. Some sections are unclear or incomplete. (3-4)	The lab record is poorly organized, with missing or unclear sections. (1-2)

	Transitions between sections are smooth. (7-8)			

Rubrics for Practical continuous assessment(Test/SEE)

Performance Indicators	Excellent	Very Good	Good	Satisfactory
Fundamental Knowledge (15)(Viva) (PO1)	The student has well depth knowledge of the topics related to the course (13-15)	Student has good knowledge of some of the topics related to course (09-12)	Student is capable of narrating the answer but not capable to show in depth knowledge (04-08)	Student has not understood the concepts clearly (01-03)
Implementation (20) (PO3 & PO8)	Student is capable of implementing the design with best suitable algorithm considering optimal solution. (17-20)	Student is capable of implementing the design with best suitable algorithm and should be capable of explaining it (13-16)	Student is capable of implementing the design with proper explanation. (06-12)	Student is capable of implementing the design. (01-05)
Result &Analysis (15) (PO4)	Student is able to run the program on various cases and compare the result with proper analysis. (13-15)	Student will be able to run the program for all the cases. (09-12)	Student will be able to run the code for few cases and analyze the output. (04-08)	Student will be able to run the program but not able to analyze the output. (01-03)

Rubrics for SEE / CIE Test:

- To pass the **CIE component**, a student must secure **a minimum of 40% of 50 marks**, i.e., **20 marks**.
- To pass the **SEE component**, a student must secure **a minimum of 35% of 50 marks**, i.e., **18 marks**.
- A student is deemed to have **successfully completed the course** if the **combined total of CIE and SEE is at least 40 out of 100 marks**.

1.Write a C program to **compute the roots of a quadratic equation** by accepting the coefficients **a**, **b**, and **c** from the user. The program should calculate the discriminant and determine the nature of the roots (real and distinct, real and equal, or complex), then compute and display the roots with appropriate messages.

Input: Three space-separated floating-point numbers representing the coefficients a, b, and c ($-10^3 \le a, b, c \le 10^3$ and $a \ne 0$).

Output: A message indicating the type of roots, followed by the computed roots rounded to two decimal places. If the roots are complex, display both real and imaginary parts.

```
#include <stdio.h>
#include <math.h>

int main() {
   float a, b, c;
   float discriminant, realPart, imagPart, root1, root2;

   // Input
```

```
    printf("Enter coefficients a, b and c: ");
    scanf("%f %f %f", &a, &b, &c);

    // Calculate discriminant
    discriminant = b * b - 4 * a * c;

    if (discriminant > 0) {
        // Real and distinct roots
        root1 = (-b + sqrt(discriminant)) / (2 * a);
        root2 = (-b - sqrt(discriminant)) / (2 * a);
        printf("Roots are real and distinct.\n");
        printf("Root1 = %.2f, Root2 = %.2f\n", root1, root2);
    }
    else if (discriminant == 0) {
        // Real and equal roots
        root1 = root2 = -b / (2 * a);
        printf("Roots are real and equal.\n");
        printf("Root1 = Root2 = %.2f\n", root1);
    }
    else {
        // Complex roots
        realPart = -b / (2 * a);
        imagPart = sqrt(-discriminant) / (2 * a);
        printf("Roots are complex.\n");
        printf("Root1 = %.2f + %.2fi\n", realPart, imagPart);
        printf("Root2 = %.2f - %.2fi\n", realPart, imagPart);
    }

    return 0;
}
```

2.You are developing a billing software module for an electricity supply company **"Green Volt Power Services"**. The program will help the company's customer service department quickly calculate monthly electricity bills for individual customers.

The billing follows the company's tariff policy:

- **i)** **First 200 units:** ₹0.80 per unit
- **ii)** **Next 100 units (201–300):** ₹0.90 per unit
- **iii)** **Beyond 300 units:** ₹1.00 per unit
- **iv)** **Minimum meter charge:** ₹100 for all customers (regardless of usage)
- **v)** **Surcharge:** If the energy cost (excluding meter charge) exceeds ₹400, add a **15% surcharge** on that cost.

Input: A string representing the **customer's name** (no spaces)**and** An integer representing the **units consumed**
Output :Consumer name and bill amount

Algorithm

1. Start
2. Read customer name and units consumed.
3. Initialize bill = 0.

4. If units ≤ 200 → bill = units × 0.80.
5. Else if units ≤ 300 → bill = (200 × 0.80) + (units – 200) × 0.90.
6. Else → bill = (200 × 0.80) + (100 × 0.90) + (units – 300) × 1.00.
7. If bill > 400 → add surcharge = 15% of bill.
8. Add minimum meter charge ₹100.
9. Display final bill.
10. Stop

```
#include <stdio.h>

int main() {
   char name[50];
   int units;
   float bill, surcharge = 0;

   // Input
   printf("Enter customer name (no spaces): ");
   scanf("%s", name);
   printf("Enter units consumed: ");
   scanf("%d", &units);

   // Calculate bill
   if (units <= 200) {
      bill = units * 0.80;
   } else if (units <= 300) {
      bill = (200 * 0.80) + (units - 200) * 0.90;
   } else {
      bill = (200 * 0.80) + (100 * 0.90) + (units - 300) * 1.00;
   }

   // Add surcharge if bill > 400
   if (bill > 400) {
      surcharge = 0.15 * bill;
   }

   // Add minimum meter charge
   bill = bill + surcharge + 100;

   // Output
   printf("\nCustomer: %s\n", name);
   printf("Total Bill = Rs. %.2f\n", bill);

   return 0;
}
```

3.The bank's security system needs to verify cheque numbers entered by customers. A cheque number is considered unusual if it is a **palindrome** Such numbers are rare and may be flagged for a **special security check** as they are often used for memorable or special transactions.
Input : A single positive integer n ($1 \le n \le 10^6$)
Output: It is a palindrome or It is not a palindrome

Algorithm

1. **Start**
2. Input a number n.
3. Store the original number in a variable temp.
4. Initialize rev = 0.
5. Repeat while n > 0:
    - Extract digit = n % 10
    - Update rev = rev * 10 + digit
    - Update n = n / 10
6. If rev == temp → The number is a palindrome.
7. Else → Not a palindrome.
8. **Stop**

```
#include <stdio.h>

int main() {
   int n, temp, rev = 0, digit;

   printf("Enter a number: ");
   scanf("%d", &n);

   temp = n;

   while (n > 0) {
      digit = n % 10;
      rev = rev * 10 + digit;
      n = n / 10;
   }

   if (rev == temp) {
      printf("It is a palindrome\n");
   } else {
      printf("It is not a palindrome\n");
   }

   return 0;
}
```

4. In a biometrics research lab, scientists often need to calculate the number of possible arrangements of fingerprints in a dataset. The number of arrangements for n unique fingerprints is given by the factorial of n. To make the process efficient and easier to maintain, they decide to implement the factorial calculation using a recursive function in C.
Input: A single non-negative integer n ($0 \le n \le 20$).
Output: A single line displaying Factorial of n = <value>.

Algorithm

1. **Start**
2. Input a number n.
3. If n == 0 or n == 1, return 1.
4. Else return n × factorial(n-1).
5. Display the factorial result.

6. **Stop**

```
#include <stdio.h>

// Recursive function to calculate factorial
long long factorial(int n) {
   if (n == 0 || n == 1)
      return 1;
   else
      return n * factorial(n - 1);
}

int main() {
   int n;
   printf("Enter a number: ");
   scanf("%d", &n);

   if (n < 0) {
      printf("Factorial is not defined for negative numbers.\n");
   } else {
      printf("Factorial of %d = %lld\n", n, factorial(n));
   }

   return 0;
}
```

5. In a **university library**, all book IDs are stored in ascending order in the database.
When a student enters a book ID to search, the system must **quickly** check if the book exists in the catalog and display its **position** in the list if found. If the book ID does not exist, display a message indicating the book is not available.

Input:

- First line: Integer n (number of books in the catalog, $1 \leq n \leq 10^5$)
- Second line: n space-separated sorted integers (book IDs)
- Third line: Integer key (the book ID to search)

Output:

- Book found at position <index> (0-based index) if the book exists.
- Book not found if it doesn't exist.

Algorithm

1. **Start**
2. Input number of books n.
3. Read n sorted book IDs into an array.
4. Input the key (book ID to search).
5. Initialize low = 0, high = n-1, found = -1.
6. Repeat while low ≤ high:
   - Compute mid = (low + high) / 2.
   - If arr[mid] == key → set found = mid and break.
   - Else if arr[mid] < key → set low = mid + 1.
   - Else → set high = mid - 1.
7. If found != -1, display "Book found at position <found>".

8. Else display "Book not found".
9. **Stop**

```
#include <stdio.h>

int main() {
   int n, key, low, high, mid, found = -1;

   printf("Enter number of books: ");
   scanf("%d", &n);

   int arr[n];
   printf("Enter %d sorted book IDs: ", n);
   for (int i = 0; i < n; i++) {
      scanf("%d", &arr[i]);
   }

   printf("Enter book ID to search: ");
   scanf("%d", &key);

   low = 0;
   high = n - 1;

   while (low <= high) {
      mid = (low + high) / 2;
      if (arr[mid] == key) {
         found = mid;
         break;
      } else if (arr[mid] < key) {
         low = mid + 1;
      } else {
         high = mid - 1;
      }
   }

   if (found != -1)
      printf("Book found at position %d\n", found);
   else
      printf("Book not found\n");

   return 0;
}
```

6. The central branch of **Secure Bank** receives daily transaction amounts from its various ATMs. Before generating end-of-day reports, the transaction amounts must be sorted in ascending order for easier auditing and anomaly detection.

Input:

- The first line contains an integer n ($1 \leq n \leq 10^4$) — the number of transactions.
- The second line contains n space-separated integers representing the transaction amounts.

Output:

- A single line displaying the sorted transaction amounts in ascending order, separated by spaces.

Algorithm

1. **Start**
2. Input the number of transactions n.
3. Read n transaction amounts into an array.
4. Use a sorting algorithm (Bubble Sort) to sort the array in ascending order:
    - Repeat for i = 0 to n-1:
        - For j = 0 to n-i-2:
            - If arr[j] > arr[j+1], swap them.
5. Display the sorted array.
6. **Stop**

```
#include <stdio.h>

int main() {
    int n, i, j, temp;

    printf("Enter number of transactions: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d transaction amounts: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Bubble Sort
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    printf("Sorted transaction amounts: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

7**.** In the TechText Data Processing Unit, software tools handle large volumes of text data received from different departments. However, some systems operate in low-memory embedded environments where built-in string library functions like strcmp, strcat, and strlen are not available. To ensure compatibility, engineers need to manually implement basic string operations using custom functions.

The operations required are:

1. String Comparison – to check if two pieces of data are identical.
2. String Concatenation – to merge two text inputs for further processing.
3. String Length Calculation – to find the size of incoming text data.

Input:

- First line: an integer choice (1 for Compare, 2 for Concatenate, 3 for Length)
- For choice 1 or 2: read two strings str1 and str2
- For choice 3: read one string str

Output:

- For Compare: print "Strings are equal" or "Strings are not equal"
- For Concatenate: print the concatenated string
- For Length: print "Length = <value>"

Algorithm

1. **Start**
2. Input a choice (1 → Compare, 2 → Concatenate, 3 → Length).
3. If choice = 1 (Compare):
    - Input two strings str1, str2.
    - Traverse character by character.
    - If any mismatch, print "Strings are not equal".
    - If all match, print "Strings are equal".
4. If choice = 2 (Concatenate):
    - Input two strings.
    - Copy first string to result.
    - Append second string to result.
    - Print concatenated string.
5. If choice = 3 (Length):
    - Input a string.
    - Count characters until '\0'.
    - Print length.
6. **Stop**

```
#include <stdio.h>

// Function to find string length
int strLength(char str[]) {
   int len = 0;
   while (str[len] != '\0')
      len++;
   return len;
}

// Function to compare two strings
int strCompare(char str1[], char str2[]) {
   int i = 0;
   while (str1[i] != '\0' && str2[i] != '\0') {
      if (str1[i] != str2[i])
         return 0; // not equal
      i++;
   }
   return (str1[i] == '\0' && str2[i] == '\0');
}
```

```
// Function to concatenate two strings
void strConcat(char str1[], char str2[], char result[]) {
    int i = 0, j = 0;
    while (str1[i] != '\0') {
        result[i] = str1[i];
        i++;
    }
    while (str2[j] != '\0') {
        result[i] = str2[j];
        i++;
        j++;
    }
    result[i] = '\0';
}

int main() {
    int choice;
    char str1[100], str2[100], result[200];

    printf("Enter your choice (1-Compare, 2-Concatenate, 3-Length): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Enter first string: ");
        scanf("%s", str1);
        printf("Enter second string: ");
        scanf("%s", str2);
        if (strCompare(str1, str2))
            printf("Strings are equal\n");
        else
            printf("Strings are not equal\n");
    }
    else if (choice == 2) {
        printf("Enter first string: ");
        scanf("%s", str1);
        printf("Enter second string: ");
        scanf("%s", str2);
        strConcat(str1, str2, result);
        printf("Concatenated string: %s\n", result);
    }
    else if (choice == 3) {
        printf("Enter a string: ");
        scanf("%s", str1);
        printf("Length = %d\n", strLength(str1));
    }
    else {
        printf("Invalid choice\n");
    }

    return 0;
}
```

8. In a **railway reservation system**, the seat allocation data for each train is stored in a matrix format.

- The first matrix contains the **number of seats booked** for each coach on different routes.
- The second matrix contains the **fare per seat** for each coach type on each route.

To calculate the **total fare collected** for every coach-route combination, the system multiplies these two matrices. Write a program using functions to perform this matrix multiplication. The program should:

- Read the dimensions and elements of two matrices.
- Verify that the number of columns in the first matrix equals the number of rows in the second matrix (matrix multiplication rule).
- if valid, multiply the matrices and display the resulting matrix.
- If invalid, display "Matrix multiplication not possible".

Input:

- First line: two integers m1 and n1 ($1 \leq m1, n1 \leq 100$), the dimensions of the first matrix.
- Next m1 lines: each containing n1 space-separated integers (booked seats matrix).
- Next line: two integers m2 and n2 ($1 \leq m2, n2 \leq 100$), the dimensions of the second matrix.
- Next m2 lines: each containing n2 space-separated integers (fare per seat matrix).

Output:

- If multiplication is possible (n1 == m2), print the product matrix with m1 rows and n2 columns, each row on a new line with space-separated integers.
- Otherwise, print "Matrix multiplication not possible".

Algorithm

1. **Start**
2. Input dimensions m1, n1 of the first matrix.
3. Read elements of the first matrix.
4. Input dimensions m2, n2 of the second matrix.
5. Read elements of the second matrix.
6. If n1 != m2, print "Matrix multiplication not possible" and stop.
7. Else, initialize result matrix with zeros.
8. For each row i of first matrix and column j of second matrix:
    - Compute $C[i][j] = \Sigma (A[i][k] \times B[k][j])$ for k = 0 to n1-1.
9. Display the result matrix.
10. **Stop**

```
#include <stdio.h>

int main() {
   int m1, n1, m2, n2, i, j, k;

   // Input dimensions and first matrix
   printf("Enter dimensions of first matrix (m1 n1): ");
   scanf("%d %d", &m1, &n1);

   int A[m1][n1];
   printf("Enter elements of first matrix:\n");
   for (i = 0; i < m1; i++) {
      for (j = 0; j < n1; j++) {
         scanf("%d", &A[i][j]);
```

```
        }
    }

    // Input dimensions and second matrix
    printf("Enter dimensions of second matrix (m2 n2): ");
    scanf("%d %d", &m2, &n2);

    int B[m2][n2];
    printf("Enter elements of second matrix:\n");
    for (i = 0; i < m2; i++) {
        for (j = 0; j < n2; j++) {
            scanf("%d", &B[i][j]);
        }
    }

    // Check multiplication possibility
    if (n1 != m2) {
        printf("Matrix multiplication not possible\n");
        return 0;
    }

    int C[m1][n2];

    // Initialize result matrix
    for (i = 0; i < m1; i++) {
        for (j = 0; j < n2; j++) {
            C[i][j] = 0;
        }
    }

    // Perform multiplication
    for (i = 0; i < m1; i++) {
        for (j = 0; j < n2; j++) {
            for (k = 0; k < n1; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    // Display result
    printf("Resultant matrix:\n");
    for (i = 0; i < m1; i++) {
        for (j = 0; j < n2; j++) {
            printf("%d ", C[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

9. Use structures and write a program to manage performance data for cricket academy players, storing their ID, name, score, and skill level. The program should read details for N players, compute the overall average score, assign skill levels based on performance ranges, display the average and category-wise counts, list all players with their details, and identify the top three performers.

Input:

The first line contains an integer N representing the number of players ($1 \leq N \leq 100$).

The next N lines each contain three values: an integer player_id (unique player ID), a string player_name (the player's full name, underscores allowed for spaces), and an integer score (runs scored in the match, $0 \leq score \leq 100$).

Output:

The program should display:

1. The overall average score (rounded to two decimal places).
2. The number of players in each skill level category (Elite, Advanced, Intermediate, Beginner).
3. A detailed list of all players showing ID, name, score, and skill level.
4. The top three performers with their names and scores in descending order.

Algorithm

1. **Start**
2. Input number of players N.
3. For each player:
    - Read id, name, and score.
    - Store details in a structure array.
4. Compute total and average score.
5. Assign skill levels:
    - **Elite**: score ≥ 80
    - **Advanced**: 60 ≤ score < 80
    - **Intermediate**: 40 ≤ score < 60
    - **Beginner**: score < 40
6. Count how many players fall in each category.
7. Display average score and counts.
8. Display all players with ID, Name, Score, and Skill.
9. Sort players by score (descending).
10. Display top 3 performers.
11. **Stop**

```
#include <stdio.h>
#include <string.h>

struct Player {
   int id;
   char name[50];
   int score;
   char skill[20];
};

int main() {
   int N, i, j;
   printf("Enter number of players: ");
   scanf("%d", &N);
```

```
struct Player players[N];
int total = 0;

// Input player details
for (i = 0; i < N; i++) {
    printf("Enter ID, Name (use _ for spaces), and Score: ");
    scanf("%d %s %d", &players[i].id, players[i].name, &players[i].score);
    total += players[i].score;

    // Assign skill level
    if (players[i].score >= 80)
        strcpy(players[i].skill, "Elite");
    else if (players[i].score >= 60)
        strcpy(players[i].skill, "Advanced");
    else if (players[i].score >= 40)
        strcpy(players[i].skill, "Intermediate");
    else
        strcpy(players[i].skill, "Beginner");
}

float avg = (float) total / N;
printf("\nOverall Average Score = %.2f\n", avg);

// Count categories
int elite = 0, adv = 0, inter = 0, beg = 0;
for (i = 0; i < N; i++) {
    if (strcmp(players[i].skill, "Elite") == 0) elite++;
    else if (strcmp(players[i].skill, "Advanced") == 0) adv++;
    else if (strcmp(players[i].skill, "Intermediate") == 0) inter++;
    else beg++;
}

printf("Elite: %d, Advanced: %d, Intermediate: %d, Beginner: %d\n",
       elite, adv, inter, beg);

// Display all players
printf("\nPlayer Details:\n");
for (i = 0; i < N; i++) {
    printf("ID: %d, Name: %s, Score: %d, Skill: %s\n",
           players[i].id, players[i].name, players[i].score, players[i].skill);
}

// Sort players by score (descending)
struct Player temp;
for (i = 0; i < N - 1; i++) {
    for (j = i + 1; j < N; j++) {
        if (players[j].score > players[i].score) {
            temp = players[i];
            players[i] = players[j];
            players[j] = temp;
        }
```

```
        }
    }

    // Display top 3 performers
    printf("\nTop Performers:\n");
    for (i = 0; i < N && i < 3; i++) {
        printf("%s with Score: %d\n", players[i].name, players[i].score);
    }

    return 0;
}
```

10. In a **manufacturing quality control system**, engineers measure the **lengths** of n machine parts in milimeters to ensure they meet design specifications. To analyze production consistency, the system must compute:

1. The **total length** of all measured parts.
2. The **average length** (mean).
3. The **standard deviation** to determine variation in part sizes.

Write a C program using **pointers** to store the measurements in an array and perform all calculations using **pointer arithmetic**.

Input:

- First line: An integer n ($1 \leq n \leq 1000$) — number of parts measured.
- Second line: n space-separated real numbers representing the lengths of the parts (in mm).

Output:

- Sum = <value>
- Mean = <value>
- Standard Deviation = <value>(All values rounded to two decimal places.

Algorithm

1. **Start**
2. Input number of parts n.
3. Read n part lengths into an array.
4. Use a pointer to traverse the array and compute the **sum**.
5. Compute the **mean = sum / n**.
6. Traverse again with pointer to calculate variance:
    - For each element: (value – mean)²
    - Add to variance sum.
    - Divide by n to get variance.
7. Compute **standard deviation = sqrt(variance)**.
8. Display Sum, Mean, and Standard Deviation (rounded to 2 decimals).
9. **Stop**

```
#include <stdio.h>
#include <math.h>

int main() {
    int n, i;
    double sum = 0, mean, variance = 0, stdDev;

    printf("Enter number of parts: ");
```

```
    scanf("%d", &n);

    double arr[n];
    printf("Enter %d part lengths: ", n);
    for (i = 0; i < n; i++) {
        scanf("%lf", &arr[i]);
    }

    // Using pointer for sum
    double *ptr = arr;
    for (i = 0; i < n; i++) {
        sum += *(ptr + i);
    }

    mean = sum / n;

    // Calculate variance
    for (i = 0; i < n; i++) {
        variance += pow(*(ptr + i) - mean, 2);
    }
    variance /= n;

    stdDev = sqrt(variance);

    printf("Sum = %.2f\n", sum);
    printf("Mean = %.2f\n", mean);
    printf("Standard Deviation = %.2f\n", stdDev);

    return 0;
}
```